东 南 大 学 考 试 卷（2016）

1. 说说你对 NP 完全性的理解。（8 分）

见 2015 年秋

2. 对于有些问题为何需要设计近似算法求解。（7 分）

1. 问题是 NP 完全或 NP 困难：这些问题在现有理论下无法在多项式时间内找到精确解，通过近似算法，可以在可接受的时间内找到一个次优解，且次优解与最优解的误差有严格界定

2. 精确算法可能需要大量的计算资源（如时间、内存），对于大规模问题，这种资源消耗不可接受

3.许多近似算法运行时间为多项式时间，例如 O(n \log n) 或 O(n^2) ，相比指数时间的精确算法更高效

3. 如何证明一个贪心算法能够得到最优解？（10 分）

证明一个贪心算法能够得到最优解，通常包括以下步骤：

Step 1: 验证最优子结构性质

证明问题的最优解可以分解为一个子问题的最优解以及当前问题的一个贪心选择。

- **分析问题规模缩小后的子问题。**
- **说明贪心选择不会影响子问题的最优性。**

Step 2: 验证贪心选择性质

证明贪心算法每一步做出的选择，与一个全局最优解中的某一步选择一致。

- **假设存在一个最优解，其选择与贪心选择不同。**
- **通过调整该解（换成贪心选择），说明调整后解仍然是最优解，从而验证贪心选择的正确性。**

Step 3: 总结证明贪心算法的最优性

结合两点性质，说明通过贪心选择可以一步步递归地构造全局最优解，因此贪心算法是正确的。

4. 基于两两关键字比较的排序算法，时间复杂度的下界为多少？为什么？（15 分）

基于**两两关键字比较**的排序算法（如冒泡排序、选择排序、归并排序、快速排序等），其时间复杂度的**下界为** Ω(nlogn) 原因：

1. 比较排序的决策树模型

在基于比较的排序算法中，算法通过两两比较来确定序列中元素的顺序关系。我们可以将整个排序过程建模为一个**决策树**：

- 每次比较是一个分支操作，决策树的每个叶子节点表示一种可能的排列结果。
- **节点数：** 对于 n 个元素，共有 $n!$ 种可能的排列，因此决策树的叶子节点数为 $n!$。

2. 决策树的深度分析

- 决策树的深度对应于排序算法所需的比较次数。也就是说，排序算法的最坏时间复杂度是决策树的高度。
- **完全二叉树的最小高度：** 若决策树有 L 个叶子节点，其最小高度 h 满足：

$$L \leq 2^h \quad \Rightarrow \quad h \geq \log_2 L$$

- 对于 $n!$ 个排列，决策树的最小高度为：

$$h \geq \log_2(n!)$$

3. $\log_2(n!)$ 的下界

利用对数阶近似公式（斯特林公式），我们可以估算 $\log_2(n!)$ 的值：

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

取对数：

$$\log_2(n!) \approx \log_2\left(\sqrt{2\pi n}\right) + n\log_2\left(\frac{n}{e}\right)$$

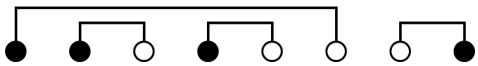
忽略低阶项后，有：

$$\log_2(n!) \sim n\log_2 n - n$$

因此，比较排序算法的最坏时间复杂度下界为：

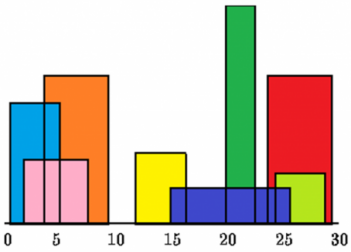
$$\Omega(n\log n)$$

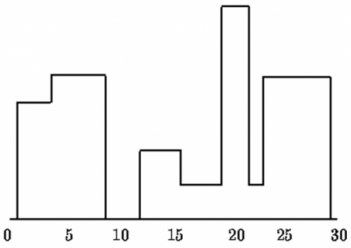
5. 给定直线上 n个白点和 n个黑点，相邻两个点之间距离均为 1，请设计一个算法将每个白点与另一黑点相连，使得连线的总长度最小。例如，图中有 4 个白点和 4 个黑点，以图中连线方式相连，连线总长度为 1+1+1+5=8。（15 分）



算法思想：一定存在相邻不同色的点，将最左边的相邻不同色的点连接，并删除，从剩下的点中执行相同操作。

6. 给定n座建筑物，每个建筑物i是一个矩形，表示为一个三元组(a_i, b_i, h_i)，其中a_i表示建筑左下顶点，b_i表示建筑的右下顶点，h_i表示建筑的高，请设计一个$O(n\log n)$的算法求出这n座建筑物的天际轮廓。例如，左下图所示中 8 座建筑的表示分别为(1,5,11), (2,7,6), (3,9,13), (12,16,7), (14,25,3), (19,22,18), (23,29,13)和(24,28,4)，其天际轮廓如右下图所示可用 9 个水平高度 (1, 11), (3, 13), (9, 0), (12, 7), (16, 3), (19, 18), (22, 3), (23, 13)和(29,0)表示。举另一个例子，假定只有一个建筑物(1, 5, 11)，其天际轮廓为输出为(1, 11), (5, 0)。（15 分）





算法思想：类似于 merge sort 的分治算法

算法思路

利用**分治法**解决天际轮廓问题，可以高效地计算出天际轮廓。核心思想如下：

1. **分治划分：**
   - 将 n 个建筑物按照中点划分为两部分，分别求出每部分的天际轮廓。
   - 递归地继续划分，直到子问题中只包含一个建筑物。
2. **合并天际轮廓：**
   - 对两部分的天际轮廓进行**归并**，生成最终的完整天际线。
3. **复杂度分析：**
   - 每次划分为两部分，递归深度为 $O(\log n)$。
   - 合并两个部分的天际轮廓需要 $O(n)$。
   - 总体复杂度为 $O(n \log n)$。

算法步骤

1. 分治方法

1. 如果只有一个建筑物 (a, b, h)，天际轮廓为 $[(a, h), (b, 0)]$。
2. 如果有多个建筑物：
   - 将建筑物分为两部分，分别求解左侧和右侧的天际轮廓。
   - 将两部分天际轮廓合并。

2. 合并天际线

1. 逐步遍历左右两部分的天际线，按照横坐标 x 的顺序合并。
2. 维护当前的最高高度，决定当前天际线的高度值 y：
   - 如果某个 x 只在左侧或右侧出现，则直接使用该高度。
   - 如果某个 x 同时出现在左右两侧，则取两侧的最大高度。
3. 过滤掉连续重复的高度点。

```
def getSkyline(buildings):
    # 基础情况：如果只有一个建筑物
    if len(buildings) == 1:
        a, b, h = buildings[0]
        return [(a, h), (b, 0)]

    # Step 1: 分治划分
    mid = len(buildings) // 2
    leftSkyline = getSkyline(buildings[:mid])
    rightSkyline = getSkyline(buildings[mid:])

    # Step 2: 合并两个天际线
    return mergeSkyline(leftSkyline, rightSkyline)

def mergeSkyline(left, right):
    result = []
    h1, h2 = 0, 0  # 当前左右天际线的高度
    i, j = 0, 0  # 左右天际线的索引
```

```
while i < len(left) and j < len(right):
    # 比较左右天际线的 x 坐标
    if left[i][0] < right[j][0]:
        x, h1 = left[i]
        i += 1
    elif left[i][0] > right[j][0]:
        x, h2 = right[j]
        j += 1
    else:
        x, h1 = left[i]
        h2 = right[j][1]
        i += 1
        j += 1

    # 取当前高度的最大值
    maxHeight = max(h1, h2)
    if not result or result[-1][1] != maxHeight:
        result.append((x, maxHeight))

# 加入剩余的天际线点
result.extend(left[i:])
result.extend(right[j:])

return result
```

7. 给定 n个活动，活动a_i表示为一个三元组(s_i, f_i, v_i)，其中s_i表示活动开始时间，f_i表示活动的结束时间，v_i表示活动的权重。带权活动选择问题是选择一些活动，使得任意被选择的两个活动a_i和a_j执行时间互不相交，即区间$[s_i, f_i]$与$[s_j, f_j]$互不重叠，并且被选择的活动的权重和最大。请设计一种方法求解带权活动选择问题，并证明算法的正确性。（15 分）

算法思想：动态规划

Notation. Label jobs by finishing time: $f_1 \le f_2 \le \ldots \le f_n$.
Def. p(j) = largest index i < j such that job i is compatible with j.

Notation. OPT(j) = value of optimal solution to the problem consisting of job requests 1, 2, ..., j.

$$OPT(j) = \begin{cases} 0 & \text{if } j = 0 \\ \max\left\{ v_j + OPT(p(j)),\ OPT(j-1) \right\} & \text{otherwise} \end{cases}$$

```python
def weighted_activity_selection(activities):
    # Step 1: 按结束时间排序
    activities.sort(key=lambda x: x[1])  # 按 (s_i, f_i, v_i) 中的 f_i 排序

    # Step 2: 预计算 p(j)(最近的不冲突活动索引)
    def find_previous(j):
        # 用二分查找找到与 activities[j] 不冲突的最近活动
        low, high = 0, j - 1
        while low <= high:
            mid = (low + high) // 2
            if activities[mid][1] <= activities[j][0]:
                if activities[mid + 1][1] <= activities[j][0]:
                    low = mid + 1
                else:
                    return mid
            else:
                high = mid - 1
        return -1

    n = len(activities)
    p = [find_previous(j) for j in range(n)]

    # Step 3: 动态规划计算 OPT(j)
    dp = [0] * (n + 1)  # dp[i] 表示前 i 个活动的最大权重和

    for j in range(1, n + 1):
        # 注意: activities 索引从 0 开始, 而 dp 从 1 开始
        without_j = dp[j - 1]
        with_j = activities[j - 1][2] + (dp[p[j - 1] + 1] if p[j - 1] != -1 else 0)
        dp[j] = max(without_j, with_j)

    # Step 4: 反向追踪找到选取的活动集合
    selected_activities = []
    j = n
    while j > 0:
        if activities[j - 1][2] + (dp[p[j - 1] + 1] if p[j - 1] != -1 else 0) > dp[j - 1]:
            selected_activities.append(activities[j - 1])
            j = p[j - 1] + 1
        else:
            j -= 1

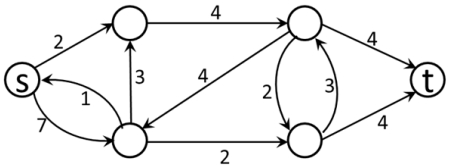
    selected_activities.reverse()
    return dp[n], selected_activities
```

1. **正确性证明：**

   - **优化子结构性质：** 对于 j 活动，如果选择它，则前面的活动必须是与其不冲突的最优解子集 (即 $OPT(p(j))$)；如果不选它，则前面的活动是 $OPT(j-1)$。

   - **子问题重叠性质：** 不同的子问题 $OPT(j)$ 会共享之前计算的结果，因此动态规划高效地解决问题。

8. 下图是一个流网络，请说明 s 到 t 的最大流值是多少？并标示一种流方式。（15 分）



最大流为 6